Eğer modern bir işletim sistemi (Linux, Windows, macOS) üzerinde çalışıyorsanız, **tamamen işletim sisteminden bağımsız, onu tamamen devre dışı bırakan** bir bellek yönetimi yapmak **teknik olarak imkansızdır.**

Bunun nedeni, modern işlemcilerin ve işletim sistemlerinin korumalı modda (Protected Mode) çalışması ve **Sanal Bellek (Virtual Memory)** mimarisidir.

Ancak, işletim sisteminin bellek yönetim mekanizmalarına (sayfalama, otomatik büyütme, malloc'un getirdiği ek yükler vb.) **en az seviyede bulaşarak**, belleğin kontrolünü **neredeyse %99 oranında kendi uygulamanıza almak** mümkündür.

Bunu nasıl yapabileceğinizi, sınırlarınızı ve yöntemlerini aşağıda bulabilirsiniz:

1. Neden Tamamen Bağımsız Olamazsınız? (Donanım ve OS Engeli)

Uygulamanız başladığında doğrudan fiziksel RAM'e (örneğin donanım adresindeki 0x004F32) erişemezsiniz.

- **MMU (Memory Management Unit):** İşlemcinin içindeki bu donanım, işletim sistemiyle ortak çalışır. Siz bir adrese yazmaya çalıştığınızda, OS bunu arka planda gerçek bir RAM adresine eşler.
- **Segmentasyon / Koruma:** Eğer işletim sisteminden izin almadan doğrudan bir RAM adresine işaretçi (pointer) atayıp yazmaya çalışırsanız, işlemci anında bir donanım kesmesi (hardware interrupt) üretir ve işletim sistemi programınızı **"Segmentation Fault" (Görsel Kilitlenme/Çökme)** ile anında öldürür.

---

2. İşletim Sistemini "En Az" Şekilde Kullanarak Manuel Yönetim Nasıl Yapılır?

İşletim sisteminden tamamen kaçamazsınız ama onu sadece **"bize büyük bir arsa ver, gerisine karışma"** demek için kullanabilirsiniz. İzlenen en yaygın ve profesyonel yöntem şudur:

Adım A: İşletim Sisteminden Büyük Bir "Ham (Raw)" Blok İstemek

Program başlarken işletim sisteminin kapısını **sadece bir kez** çalarsınız. malloc yerine, işletim sisteminin en alt seviye ham bellek tahsis fonksiyonunu çağırırsınız:

- **Linux / Unix:** mmap() sistem çağrısı kullanılır. (Örn: İşletim sisteminden tek seferde 1 GB ham sanal bellek alanı istenir).
- **Windows:** VirtualAlloc() fonksiyonu kullanılır.

Bu çağrı standart malloc gibi çalışmaz. İşletim sistemi size sadece o boyutlarda boş bir adres uzayı tanımlar ve çekilir.

Adım B: Arenalar ve Custom Allocators (Özel Dağıtıcılar) Yazmak

İşletim sisteminden aldığınız bu devasa ham bloğu (örneğin 1 GB'lık uint8_t memory_pool[1024*1024*1024] gibi düşünün) artık **tamamen kendi kurallarınızla** yönetirsiniz. İşletim sistemi bu bloğun içinde ne yaptığınızı asla bilmez ve karışmaz.

İşte bu devasa bloğu içeride manuel yönetmek için kullanılan popüler stratejiler:

1. **Arena (Bump / Stack) Allocator:**
   - Belleği sadece ileriye doğru harcarsınız. Bir pointer (offset) tutarsınız. Her bellek isteğinde pointer'ı istenen boyut kadar ileri kaydırırsınız.
   - **Özelliği:** Sıfır fragmantasyon (bölünme), muazzam hız ($O(1)$ zaman karmaşıklığı). İşiniz bitince tüm arenayı tek seferde sıfırlarsınız. Free (serbest bırakma) işlemi tek tek yapılmaz.
2. **Fixed-Size Block Pool (Sabit Boyutlu Blok Havuzu):**
   - Aldığınız büyük alanı örneğin sadece 32 byte'lık veya 1024 byte'lık eşit parçalara bölersiniz. Hangi parçaların boş olduğunu bir "bağlı liste (linked list)" veya "bit dizisi (bitmask)" ile manuel takip edersiniz.
3. **Buddy Allocator:**
   - Büyük bloğu sürekli ikiye bölerek (böl-yönet) ihtiyaca göre dinamik ama katı kurallı parçalar oluşturursunuz.

Manuel Yönetimin C Kodu ile Şematik Örneği (Linux mmap ile Arena)

```c
#include <stdio.h>
#include <sys/mman.h>
#include <unistd.h>

#define ARENA_SIZE (1024 * 1024 * 10) // 10 Megabayt ham alan

typedef struct {
    uint8_t *buffer;
    size_t buffer_size;
    size_t offset;
} Arena;

// İşletim sisteminden ham bellek alan tek fonksiyon
Arena create_arena() {
    Arena arena;
    // mmap ile OS'e diyoruz ki: "Bize 10MB yer ayır, okuma/yazma
izni ver, arkasına karışma"
    arena.buffer = mmap(NULL, ARENA_SIZE, PROT_READ | PROT_WRITE,
```

```
MAP_PRIVATE | MAP_ANONYMOUS, -1, 0);
    arena.buffer_size = ARENA_SIZE;
    arena.offset = 0;
    return arena;
}

// Tamamen manuel, OS'ten bağımsız bellek dağıtımı (Bizim
malloc'umuz)
void* arena_alloc(Arena *arena, size_t size) {
    // Hizalama (Alignment) kontrolü eklenebilir. Basitlik adına
geçiyoruz.
    if (arena->offset + size > arena->buffer_size) {
        return NULL; // Bellek bitti!
    }

    void *ptr = &arena->buffer[arena->offset];
    arena->offset += size; // Pointer'ı manuel kaydırdık
    return ptr;
}

int main() {
    Arena my_os_independent_memory = create_arena();

    // Artık bu aşamadan sonra işletim sistemi tamamen devre dışı.
    // Aşağıdaki işlemler tamamen bizim belirlediğimiz pointer
matematiği ile dönüyor.
    int *my_array = (int*)arena_alloc(&my_os_independent_memory,
100 * sizeof(int));
    char *my_text = (char*)arena_alloc(&my_os_independent_memory,
50 * sizeof(char));

    if (my_array) {
        my_array[0] = 42;
        printf("İlk eleman: %d\n", my_array[0]);
    }

    // Program kapanırken OS'e arsayı geri iade ediyoruz
    munmap(my_os_independent_memory.buffer, ARENA_SIZE);
    return 0;
}
```

Özetle;

Bir işletim sistemi üzerindeyseniz, **başlangıçta alanı tapulamak (almak) için** işletim sisteminin kapısını (mmap/VirtualAlloc ile) çalmak **zorundasınız**. Ancak bu alanı aldıktan sonra projenin çalışma süresi (runtime) boyunca bellek ayırma, silme, bölme ve yer değiştirme işlemlerini işletim sistemine hiç haber vermeden, tamamen kendi yazdığınız pointer algoritmalarıyla **%100**

manuel yönetebilirsiniz. Oyun motorları (Örn: Unreal Engine) ve yüksek performanslı veri tabanları (Örn: MySQL, PostgreSQL) tam olarak bu mantıkla çalışır.

Bu seviyede bir manuel bellek yönetimi mimarisini (örneğin yukarıdaki Arena yapısını) **hangi amaçla** kullanmayı düşünüyorsunuz? Yüksek performanslı bir **oyun motoru/grafik uygulaması** mı, yoksa veri yapılarını çok hızlı işleyeceğiniz bir **algoritma** mı geliştiriyorsunuz? Amacınıza göre **hizalama (alignment)** veya **bellek fragmantasyonu çözümleri** üzerine konuşabiliriz.